TECHNICAL DESIGN · STARTING STAGE

Qlik-to-SQL Reconstruction: Technical Design

How the first stage of the Agentic Dataset Context Engine is built: architecture, agents, contracts, data flows, validation, technology and delivery.

Document	Technical design, companion to the Starting-Stage Proposal
Prepared for	Data & Engineering Leadership and technical reviewers
Date	October 2026
Status	Draft for review
Based on	Architecture v0.8 and the Qlik-first build plan v0.3

Contents

1. Purpose and scope
2. Design principles
3. Inputs and outputs
4. System architecture
5. End-to-end pipeline
6. Agents in detail
7. Qlik script processing
8. Qlik-to-SQL translation
9. Source resolution and QVD chains
10. Validation and reconciliation
11. Knowledge graph
12. Data access: Dremio
13. AI model usage
14. Human in the loop
15. Orchestration and events
16. Contracts
17. The migration pack
18. Security, privacy and identity
19. Environments and deployment
20. Observability and evaluation
21. Technology stack
22. Repository layout
23. Delivery plan
24. Success measures
25. Risks and mitigations
26. Open questions
27. What follows the starting stage
28. References

1. Purpose and scope

This document describes **how** the starting stage of the Agentic Dataset Context Engine (ADCE) is built. The Starting-Stage Proposal describes *what* it delivers and why. Here we set out the architecture, every agent and component involved, the data contracts between them, the validation method, the technology, the environments and the delivery plan.

In scope for the starting stage

- Reading QlikView and Qlik Sense load scripts, QlikView project folders, reload logs, QVD files and exported reports for 2–3 pilot applications, then widening to more applications.
- Translating Qlik load logic and, where definitions are available, chart measures into **Dremio SQL**.
- Resolving every Qlik source to Dremio datasets or to the Qlik application that produced a QVD file.
- Executing the SQL in Dremio and validating it against independent evidence (section 10).
- Recording all results as knowledge with evidence, trust level and history (section 11).
- Human review of ambiguities and approvals in a workbench (section 14).
- Exporting a migration pack per application (section 17).
- Running locally first, then in a staging environment on Google Cloud (section 19).

Out of scope for the starting stage (later stages, not dropped)

- Live validation against the Qlik engine (needs server access; Mode A2).
- Reconstructing reports for which no BI logic exists (inference from outputs; Mode B).
- Full dataset profiling, matching, estate analysis and the data dictionary.
- Designing a new data model (star schema, normalized model or curated Dremio views).
- Changing or re-pointing any Qlik application, or writing anything to Dremio.

Everything built in the starting stage is part of the final product. Early components are lean versions **behind their final interfaces**, so later stages extend them rather than replace them.

2. Design principles

Principle	What it means in practice
Deterministic first, AI second	Parsing, rule-based translation, SQL validation and data comparisons are code. AI models handle the long tail: unusual statements, ambiguous semantics and mismatch diagnosis. Every AI output is re-parsed and re-checked by code.
Evidence for every result	Every translated table or figure carries a validation level (V0–V6) and the evidence behind it. Nothing is called correct without a level.
Nothing silently dropped	A statement the parser cannot handle becomes a flagged item, never a gap. An unresolved variable or source goes to review, never to a guess.
Reproduce what Qlik reads	If Qlik reads a copy or a dump, the SQL reads that same dataset. A better (golden) source is flagged, never silently substituted.
Read-only towards client systems	Dremio and Qlik artifacts are only read. No writes, no reloads, no changes to applications.
No row-level data to AI models	Models see scripts, schemas, view definitions and aggregate differences only, enforced centrally in the model gateway.

Principle	What it means in practice
Repeatable results	AI responses are memoized by input, model and prompt version, so re-running on the same inputs regenerates the same outputs.
Lean but final interfaces	Early components implement the final contracts, so nothing is thrown away when the full platform arrives.
Built to ship	Every external dependency (storage, secrets, AI endpoint, data source, identity) sits behind an interface chosen by configuration.

3. Inputs and outputs

Inputs

Input	Format	Used for
Qlik Sense load scripts	Text / .qvs, or extracted from app files	Transformation logic, sources, variables
QlikView applications	-prj project folders (script + per-object XML)	Load logic, chart expressions, master items
Reload logs	Text logs per reload	Variable values as executed; rows loaded per table (V3)
QVD files	Binary with an XML header	Lineage (creating app, source statement), row counts (V3), column-level data (V4)
Exported reports	Excel, PDF	Reference figures for visuals (V5)
Dremio catalog	Spaces, sources, datasets, columns, view SQL definitions, wikis, tags	Source resolution, lineage below Qlik, grain
Dremio data	Read through bounded pushdown queries	Execution and validation (V2–V5)
Connection mapping	Qlik data-connection names → Dremio sources (reviewed)	Source resolution

Outputs

Output	Format	Consumer
Dremio SQL per Qlik table and per measure	SQL files (CTE chains)	Report owners, migration teams
Lineage map	Graph view, JSON (OpenLineage-compatible), Markdown	Data owners, auditors
Validation report	Markdown and Excel	Report owners, sponsors
Review record and open items	CSV, Markdown	Report owners
Knowledge facts	Fact log (Postgres) and graph projection	All later ADCE stages
Migration pack per application	Versioned folder (section 17)	Clients and internal teams

4. System architecture

The starting stage uses the target ADCE architecture, with only the components the Qlik-to-SQL path needs switched on. Agents compute on data through Dremio and record knowledge only through the knowledge graph service (**kg-mcp**), which enforces provenance, trust and access rules.

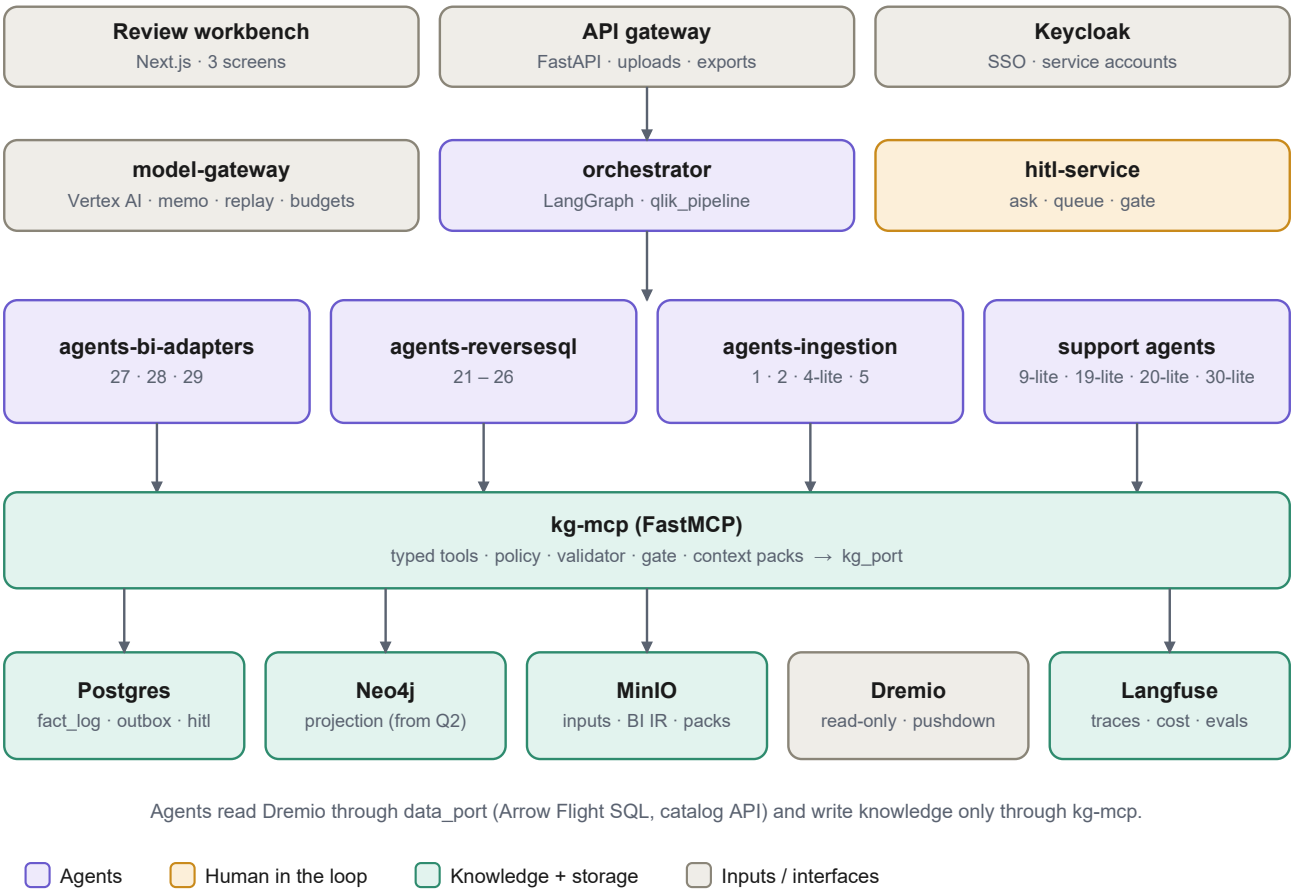


Figure 1. Components and deployable services in the starting stage.

Layer	Components in the starting stage	Added in later stages
Interfaces	Review workbench (3 screens), API gateway, Keycloak (dev realm, service accounts)	Full workbench, external MCP access, catalog exports
Ingestion	Intake router (Qlik file classes), BI extractor, Scope registrar and Schema parser (Dremio crawl), Entity linker (lite)	Document extractor, full linker, data-file uploads
Control	Orchestrator (qlik_pipeline subgraph), HITL service (Ask, Queue, Gate), model gateway (Vertex AI, memoization, replay, budgets)	All subgraphs, Seed/Correct UX, Presidio masking
Agents	Reverse-SQL (21–26, translate mode), BI adapters (27–29), Key discoverer (lite), Lineage (lite), Publisher (lite), Equivalence (lite)	Profiling, comparison and estate, definitions, inference mode, data model designer
Knowledge	kg_port over the Postgres fact log; Neo4j projection from week 6	Hybrid search, full resolution, all acceptance requirements
Storage	Postgres, MinIO, Dremio (read-only), DuckDB for files, Langfuse	Redis Streams event bus

Deployable services

Service	Responsibility
api-gateway	Uploads, exports, workbench API, token validation
workbench	Review UI (Next.js)

Service	Responsibility
orchestrator	LangGraph supervisor with the qlik_pipeline subgraph; dispatches agent jobs
hitl-service	Ask/Queue/Gate items, decisions, reuse of decisions as examples
model-gateway	All AI calls: routing, structured outputs, caching, memoization, replay, budgets, privacy policy
kg-mcp	Knowledge tools over kg_port (fact log + projection), policy, validator, gate
agents-ingestion	Intake router (1), Schema parser (2), Entity linker lite (4), Scope registrar (5)
agents-bi-adapters	BI extractor (27), Script translator (28), Expression translator (29)
agents-reversesql	Report parser (21) to Reconciler (26)
support agents	Key discoverer lite (9), Lineage lite (19), Publisher lite (20), Equivalence lite (30), deployed with their group workers

5. End-to-end pipeline

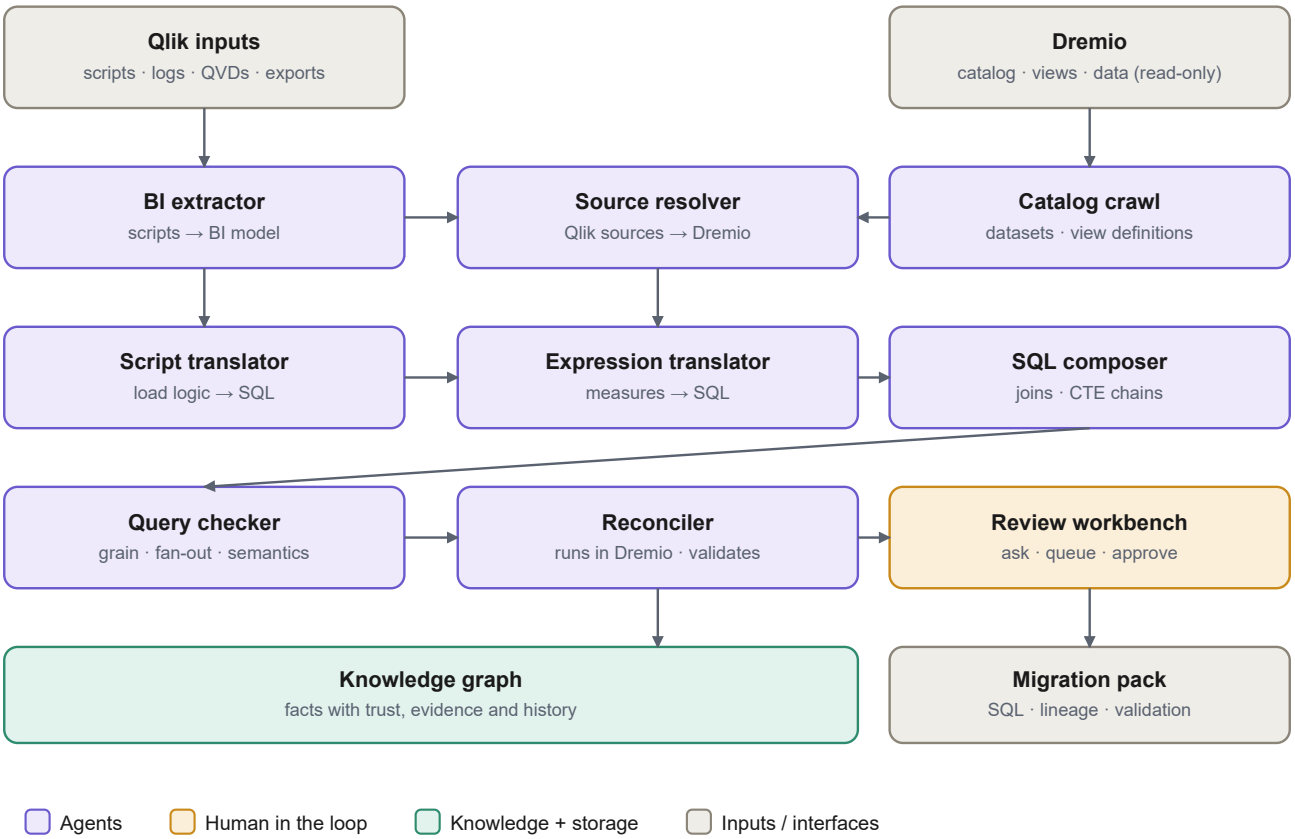


Figure 2. The Qlik-to-SQL pipeline. Every step records facts with evidence; ambiguities go to human review.

Step	Agents	What happens	Event emitted
1. Intake	1	Files are stored in MinIO under the application's scope and classified (script, project folder, app file, reload log, QVD, export, Dremio connection).	upload.classified[bi_app]
2. Catalog	5, 2	The Dremio catalog is crawled: spaces, datasets, columns, view SQL definitions (parsed into lineage).	dataset.registered
3. Extract	27	Scripts are split and parsed into the BI IR; variables resolved; QVD headers and reload logs read.	bi_ir.ready

Step	Agents	What happens	Event emitted
4. Resolve	4-lite, 5	Each source mapped to a Dremio dataset or a producer application; uncertain mappings queued.	scope_link facts
5. Grain	9-lite	Uniqueness of candidate keys checked in Dremio for every dataset Qlik reads.	primary_key facts
6. Translate	28, 29	Load statements become CTEs; measures become SQL expressions via ExprAST.	bi_transform.ready, bi_metric_def.ready
7. Compose	22, 23, 24	Measures mapped, joins planned from the Qlik model and checked against Dremio, SQL composed per table and visual.	sql.candidates
8. Check	25	Dremio dialect validity, grain and fan-out, set-analysis and TOTAL/Aggr rules; repair if needed.	sql.checked
9. Reconcile	26, 21	Executed in Dremio; compared with reload logs, QVDs and exports; mismatches diagnosed.	report_sql (with level)
10. Review	HITL	Ask, Queue and Gate items resolved in the workbench.	fact.confirmed
11. Lineage and export	19-lite, 20-lite	Lineage assembled; migration pack written.	pack.published

6. Agents in detail

Every agent, including the deterministic ones, implements the same runtime contract: receive an event, read a context pack from kg-mcp, do its work, propose facts with evidence, and emit a trace with cost and timing. Kinds: **D** deterministic (no AI), **H** hybrid (deterministic gate, then AI), **L** AI-led. Numbers match the full ADCE agent catalog.

```
on_event(evt)  -> ctx    = kg.get_context_pack(task, token_budget)
               -> result = self.work(ctx, data_port, gateway)     # code, AI, or both
               -> for p in result.proposals: kg.propose_fact(p, evidence)
               -> emit trace (Langfuse): agent@version, prompt_version, model, tokens, cost, latency
```

6.1 BI source-adapter agents

Agent 27: BI extractor

Field	Detail
Kind	D (deterministic)
Role	Turns Qlik artifacts into the tool-neutral BI IR (section 16.2).
Inputs	Qlik Sense load scripts (text/.qvs), QlikView -prj XML (script and per-object expressions, master items), Qlik Sense app files where readable offline, reload logs, QVD headers.
Processing	Statement splitting and parsing (section 7); variable resolution from reload logs and LET/SET; include files inlined with spans; QVD header XML read (CreatorDoc, NoOfRecords, LineageInfo); STORE and LOAD FROM *.qvd recorded to build QVD chains.
Writes	BIApp, Visual and Metric nodes; QVD datasets; READS/WRITES links; bi_model_link facts (documented trust); BI IR artifact in MinIO.
KG tools	register_entity, add_context, get_source_context
Triggered by	upload.classified[bi_app]; bi_inputs.received (new files for a registered application)
Human review	Confirm application scope; confirm cross-application QVD linkage; Ask for unresolved variables.

Field	Detail
Failure handling	Unparseable statements become kind: other with raw text and span, flagged and counted. Never writes, reloads or saves Qlik applications.
Later	Qlik Engine API extraction (OpenDoc, GetScript, GetTablesAndKeys, GetProperties, GetHyperCubeData) when server access exists.

Agent 28: BI script translator

Field	Detail
Kind	H (rules first, AI for the long tail)
Role	Translates load-script statements into Dremio SQL fragments (CTE chains).
Processing	Embedded SQL SELECT transpiled from its source dialect to Dremio with sqlglot; Qlik transforms mapped by the rule table (section 8.1); unmapped statements sent to Sonnet 5.5 with the statement, its inputs' schemas and similar confirmed translations; every AI output re-parsed by sqlglot and rejected if invalid.
Writes	bi_transform facts (statement → SQL fragment), lineage facts (DERIVED_FROM, documented), source-column mappings.
KG tools	get_context_pack, get_lineage, propose_fact
Triggered by	bi_ir.ready
Human review	Statements marked untranslated go to Queue.
Failure handling	Statements it cannot translate stay flagged with reasons; downstream tables depending on them are marked blocked rather than guessed.

Agent 29: BI expression translator

Field	Detail
Kind	H
Role	Translates chart measures and dimensions (inline or master items) into SQL expressions.
Processing	Parse expression to ExprAST (set modifiers, TOTAL, Aggr, Above as typed nodes); assign construct tier T1/T2/T3; rule mapping for T1–T2; Sonnet 5.5 for T2 edge cases and Opus 5.5 for T3; output re-checked.
Writes	bi_metric_def facts; definition seeds from master-item labels and descriptions (documented).
KG tools	get_context_pack, get_feedback_history, propose_fact
Triggered by	bi_ir.ready, bi_transform.*
Human review	Ask when an expression depends on unresolved variables.
Failure handling	T3 constructs that cannot be translated are reported as not resolved, with the reason.

6.2 Reverse-SQL agents

Agent 21: Report parser

Field	Detail
Kind	L
Role	Reads exported reports to obtain reference figures (V5).

Field	Detail
Processing	Excel via openpyxl; PDF via Claude PDF/vision input through the gateway; extracts measures, dimension members, filters, period and figures. Matches the export to a registered Qlik visual (title, sheet, measures) and hands off to translate mode.
Writes	Report and Metric nodes; report_structure facts; reference figures as evidence (report_export).
KG tools	get_entity, register_entity
Human review	Check parsed structure.
Privacy	Report figures are aggregates and may reach the model; no underlying rows do.

Agent 22: Metric mapper

Field	Detail
Kind	H
Role	Maps each report measure and dimension to its definition and source columns.
Processing	In translate mode, mappings come directly from bi_metric_def facts (BI field → source column via script lineage). Vector search over columns and terms only for fields with no traced source.
Writes	metric_mapping facts
KG tools	search_context, get_feedback_history, propose_fact
Human review	Ask when ambiguous.

Agent 23: Join planner

Field	Detail
Kind	D
Role	Plans the joins a reconstruction needs.
Processing	Starts from bi_model_link facts (Qlik associations on same-named fields), verifies each against Dremio view joins and confirmed joins, and prefers trusted paths. Disagreements are flagged as conflicts.
Writes	Join plan (ephemeral); conflicts
KG tools	find_join_paths, run_readonly_cypher, flag_conflict
Human review	Conflicts go to the review queue.

Agent 24: SQL synthesizer

Field	Detail
Kind	H
Role	Composes the final SQL per Qlik table and per visual.
Processing	Combines translated CTE chains (with QVD producers inlined), translated measures and the join plan through ExprAST into Dremio SQL. Variant search is disabled in translate mode except for flagged ambiguities (for example NULL handling of dual values).
Writes	Candidate SQL set
KG tools	get_context_pack

Agent 25: Query checker

Field	Detail
Kind	H
Role	Rejects or repairs SQL that would give wrong numbers.
Rules	(1) Valid Dremio SQL (sqlglot). (2) Grain and fan-out: an aggregated field is aggregated at its native table grain before joining (pre-aggregated CTE), using grain facts from agent 9-lite. (3) Set-analysis override semantics against the visual's default selection state. (4) TOTAL and Aggr scope. (5) Dual-value and NULL behaviour. (6) DISTINCT across associated tables. (7) Joins exist and units match.
Repair	Opus 5.5 proposes a repair given the rule violation; the repaired SQL is re-checked from scratch.
KG tools	get_entity (keys, grain, definitions)

Agent 26: Reconciler

Field	Detail
Kind	H
Role	Executes SQL in Dremio and climbs the validation ladder (section 10).
Processing	Bounded pushdown comparisons (counts, aggregates, grouped aggregates); Iceberg time travel to the reload time where available; result caching by SQL fingerprint, dataset versions and snapshot; grouped comparisons pinpoint mismatching dimension members.
Diagnosis	Opus 5.5 receives aggregate differences only and proposes causes (filter, fan-out, NULL handling, date logic, missing producer).
Writes	report_sql facts with validation_level; validated lineage
KG tools	propose_fact
Human review	Gate: approve the final SQL.

6.3 Supporting agents (lite versions)

#	Agent	Lite behaviour in the starting stage	Full version later
1	Intake router	Deterministic classification by extension, magic bytes and content sniffing into Qlik file classes and Dremio connections	All input classes; AI for ambiguous files
2	Schema parser	Dremio datasets, columns, types; view SQL definitions parsed with sqlglot (Dremio dialect) into lineage facts	DDL, dbt YAML, JSON Schema, Avro
4	Entity linker	Qlik source → Dremio dataset via the connection mapping, view definitions and name + schema match; uncertain links to Queue	Embeddings, value patterns, AI adjudication, document mentions
5	Scope registrar	Registers Dremio spaces and folders, Qlik applications and QVDs in the tenant scope	All source kinds; documents before data
9	Key discoverer	Grain only: uniqueness of candidate keys checked by pushdown in Dremio	Full primary-key discovery with AI validation
19	Lineage inferrer	Extraction only: transforms, QVD headers, STORE/LOAD chains, view definitions	Value checks, timestamps, documents
20	Dictionary publisher	Migration pack export (section 17)	Full dictionary in all formats

#	Agent	Lite behaviour in the starting stage	Full version later
30	Equivalence detector	SQL fingerprints and schema similarity, only to resolve Qlik sources to Dremio copies	All layers, clusters, golden records

6.4 Orchestrator

A LangGraph supervisor with the **qlik_pipeline** subgraph: intake → extract → resolve → translate → compose → check → reconcile → record → review → export. It is re-entered on bi_inputs.received, fact.confirmed (a reviewer decision) and dataset.changed, re-running only affected steps. It enforces per-run token and cost budgets, and hosts Ask interrupts (never inside a parallel fan-out). State is checkpointed in Postgres.

7. Qlik script processing

7.1 Statement splitting

Qlik scripts are statement sequences terminated by semicolons, but a naive split fails. The splitter handles:

- // and /* */ comments, and REM statements
- strings and [bracketed names] that contain semicolons
- SUB … END SUB, IF / ELSEIF / ELSE / END IF, FOR … NEXT, FOR EACH, DO … LOOP blocks
- $(Include=…) and $(Must_Include=…) files, inlined with spans pointing to the original file
- dollar-sign expansion $(variable) and $(=expression)

7.2 Grammar and statement kinds

A Lark grammar for statements and expressions is grown from real scripts, using Qlik's script and chart function references. Each statement is classified:

```
sql_select   load   resident_load   preceding_load   join   keep   concatenate
mapping_load   applymap   crosstable   generic   intervalmatch   store   drop
rename   let_set   control   other      # other = not understood: kept, flagged, counted
```

7.3 Variables

1. Values recorded in the reload log (as executed) take priority.
2. LET/SET statements are evaluated statically where deterministic.
3. Seed values from a reviewer.
4. Anything still unresolved becomes an Ask item; dependent statements are marked blocked.

7.4 Control flow

- Loops with static bounds are unrolled.
- Loops over file lists or data are resolved from the reload log where it shows the iterations.
- Otherwise the construct is tier T3 and reported.

7.5 Coverage

Coverage is a first-class metric from week 1: statements parsed, statements translated, and the rate of kind: other per application. The grammar grows wherever coverage is lowest.

8. Qlik-to-SQL translation

8.1 Rule mapping (minimum supported set)

Qlik construct	Dremio SQL
Preceding LOAD / RESIDENT load	CTE chain
CONCATENATE	UNION ALL (columns aligned, missing ones NULL-filled)
LEFT / INNER / OUTER JOIN	Same join type on all same-named fields
LEFT KEEP / INNER KEEP	WHERE EXISTS semi-join
MAPPING LOAD + ApplyMap(map, key, default)	LEFT JOIN to a mapping CTE + COALESCE(value, default)
WHERE Exists(Field)	WHERE Field IN (SELECT Field FROM earlier loads), preserving load order
Peek() / Previous()	LAG() over load order (needs an explicit ordering key; otherwise T3)
CROSSTABLE / GENERIC	UNPIVOT / PIVOT, or conditional aggregation
IntervalMatch	Range join (BETWEEN)
Sum({<F={v}, G-={w}>} X)	SUM(CASE WHEN F = v AND G <> w THEN X END)
Sum(TOTAL X) / Sum(TOTAL <D> X)	SUM(SUM(X)) OVER () / OVER (PARTITION BY D)
Aggr(Sum(X), D) inside an outer aggregate	Subquery grouped by D, then the outer aggregate
Above(expr) / RangeSum(Above(expr, 0, n))	LAG / windowed SUM … ROWS BETWEEN n-1 PRECEDING AND CURRENT ROW
Count(DISTINCT F)	COUNT(DISTINCT F) at F's native table grain
$(variable)	Resolved before translation (section 7.3); unresolved → Ask
Embedded SQL SELECT	Transpiled from the source dialect to Dremio with sqlglot
Section Access	Not translated; recorded as a row-level-security requirement

8.2 Construct tiers

Tier	Constructs	Commitment
T1	Plain aggregates, filters, simple script transforms	Committed
T2	Set analysis with literal modifiers, TOTAL, Above, ApplyMap, IntervalMatch	Committed
T3	Aggr, nested or dynamic set expressions, logic driven by dollar expansion, Peek-dependent load order	Attempted; reported as not resolved when it fails

8.3 The key semantic hazard: aggregation grain

Qlik evaluates an aggregation at the grain of the table that holds the aggregated field, so a one-to-many association does not double-count. A naive SQL join followed by SUM does. The translator therefore pre-aggregates each measure at its native grain in a CTE before joining, the Query checker rejects any SQL that aggregates across a fan-out, and reconciliation on several slices catches what remains.

```
-- Naive (wrong): order amounts repeated once per order line
SELECT c.region, SUM(o.amount) FROM orders o JOIN order_lines l ON l.order_id = o.order_id
JOIN customers c ON c.customer_id = o.customer_id GROUP BY c.region;

-- Grain-aware (matches Qlik): aggregate at the orders grain first
WITH order_amounts AS (SELECT order_id, customer_id, SUM(amount) AS amount FROM orders GROUP BY 1, 2)
SELECT c.region, SUM(oa.amount) FROM order_amounts oa
JOIN customers c ON c.customer_id = oa.customer_id GROUP BY c.region;
```

8.4 Shared intermediate form: ExprAST

Both translation (from Qlik) and, in later stages, inference (from report outputs) target one intermediate expression tree, ExprAST, before SQL is generated. The Query checker's rules are written once against it. sqlglot generates the final Dremio SQL and validates it.

9. Source resolution and QVD chains

9.1 Connection mapping

Every Qlik data connection (LIB CONNECT TO, ODBC/OLEDB CONNECT, lib:// folder paths) is mapped to a Dremio source or space, or to an original system. The mapping is drafted from the scripts in week 1–2 and confirmed by a data owner.

9.2 Resolution order for each FROM target

1. Exact Dremio path through the connection mapping.
2. Dremio view definitions that wrap the named table.
3. Name and schema match (Entity linker lite).
4. Data fingerprint match (equivalence checks, from week 9).

Each resolution is recorded as a scope_link fact with evidence. Anything below high confidence goes to the review queue.

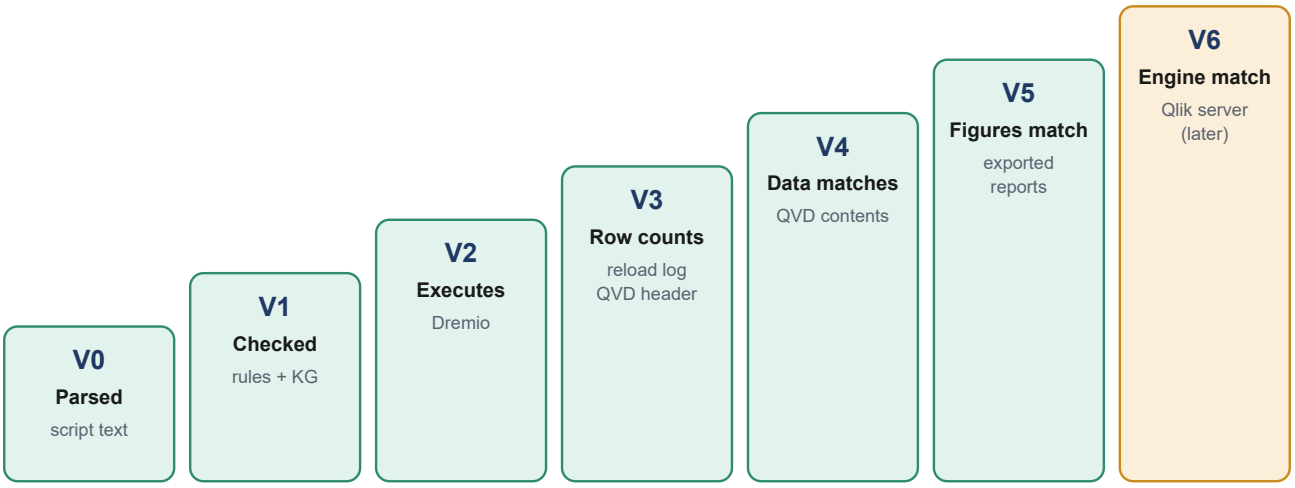
9.3 QVD chains

- Each QVD file is a dataset of kind qvd.
- Its producer is found from STORE statements and the QVD header's CreatorDoc.
- Consumers are linked through LOAD … FROM *.qvd.
- The SQL for a consumer **inlines the producer's CTE chain**, so the final SQL reads only Dremio datasets.
- If a producer's script is missing, the result is not resolved and the missing application is named in the pack's open items.

9.4 Copies and dumps in Dremio

The Dremio estate contains overlapping views and copies. The reconstruction reproduces exactly what Qlik reads, even a copy, and flags where a golden equivalent exists. It never substitutes a source silently, because that would change the numbers.

10. Validation and reconciliation



Each table and figure is labelled with the highest level it reaches. Higher levels need more evidence.

Figure 3. The validation ladder.

Level	Check	Ground truth	Comparison rule
V0 Parsed	Every statement in the BI IR; nothing skipped	Script text	100% accounted for (untranslated counted)
V1 Checked	Valid Dremio SQL; Query-checker rules pass	Rules and KG	No violation
V2 Executes	Runs within budget; expected columns and types	Dremio	Schema equals the BI IR table definition
V3 Row counts	Rows per Qlik table	Reload log lines fetched; QVD NoOfRecords	Exact, or within the drift tolerance
V4 Data	Per column: distinct count, nulls, min/max, sum; hash of a sorted key sample	QVD contents	Exact on aligned snapshots; tolerance otherwise; diffs kept
V5 Figures	Figures per visual and dimension member	Exported reports or owner figures	Exact, or within display rounding
V6 Engine	GetHyperCubeData across 3 or more selections	Qlik engine	Later, with server access

10.1 Time alignment

- Iceberg sources in Dremio are queried AT TIMESTAMP the reload time, so comparisons are exact.
- Otherwise a drift tolerance applies (default: row-count and aggregate differences of at most 0.5%, tunable per dataset), and the snapshot gap is recorded as the reason.
- Drift results are labelled "within tolerance (drift)" and never "exact".

10.2 How comparisons run

1. Each comparison is a bounded pushdown query (count, aggregates, grouped aggregates); only small result sets leave Dremio.
2. QVD-side statistics are computed locally from the QVD file with qvdrs or PyQvd.
3. Results are cached by SQL fingerprint, dataset versions and snapshot, so re-runs cost nothing until inputs change.

4 Grouped comparisons identify the dimension members that differ; the diagnoser receives these aggregate differences only.

5 Queries are labelled, concurrency-limited, time-limited and scheduled off-peak where required.

11. Knowledge graph

All results are knowledge with provenance. The system of record is an **append-only, bi-temporal fact log in Postgres**. A Neo4j projection (from week 6) supports lineage traversal and the graph view. The knowledge core builds on our in-house knowledge-graph engine, extended for data-catalog facts.

11.1 Fact log

```
CREATE TABLE fact_log (
  fact_id        uuid        NOT NULL,
  version        int         NOT NULL,
  tenant_id      text        NOT NULL,
  scope_id       text        NOT NULL,
  kind           text        NOT NULL,   -- bi_transform, bi_model_link, report_sql, scope_link, ...
  subject_ref    text        NOT NULL,   -- canonical id of the app, table, visual, dataset or column
  payload        jsonb       NOT NULL,   -- validated against the fact-kind schema
  trust          text        NOT NULL,   -- confirmed | documented | validated | inferred
  confidence     numeric,
  impact         int,                    -- number of dependent facts
  valid_from     timestamptz, valid_to timestamptz,    -- when true in the data
  recorded_at    timestamptz NOT NULL DEFAULT now(),
  invalidated_at timestamptz,                          -- when we stopped believing it
  supersedes     uuid,
  evidence       jsonb       NOT NULL,   -- [{kind, ref}]
  produced_by    text        NOT NULL,   -- agent@version
  prompt_version text, model text, run_id uuid,
  PRIMARY KEY (fact_id, version)
);
-- Insert-only: the application role has INSERT and SELECT, never UPDATE or DELETE.
CREATE TABLE outbox (id bigserial PRIMARY KEY, fact_id uuid, version int, event text,
                     payload jsonb, created_at timestamptz DEFAULT now(), published_at timestamptz);
```

11.2 Trust levels

Level	Source	Starting-stage example
Confirmed	Human decision	Reviewer approves the SQL for a table
Documented	Scripts, view definitions, QVD headers	A Qlik load statement and its translation
Validated	Checked against data	Translated table reaches V3 or higher
Inferred	AI reasoning, not yet checked	A translation of an unusual statement before validation

BI-extracted logic enters as **documented**: it describes what the report does, not that the logic is correct. It becomes **validated** only after reconciliation, and **confirmed** only through review. Disagreements create conflicts for review; nothing is overwritten. Confirmed facts never change and can only be superseded by a newer confirmation.

11.3 Fact and evidence kinds used in the starting stage

Type	Kinds
Facts	scope_link, schema_def, lineage, primary_key (grain), bi_model_link, bi_transform, bi_metric_def, metric_mapping, report_structure, report_sql (with validation_level)
Evidence	bi_script (application + statement span), bi_model (association key), qvd_header, reload_log (file + line span), view_definition, report_export (file + cell or page), dq_check, human

11.4 Graph model (projection)

```
(:Source {kind:"dremio"|"bi_server"})-[:HAS_SCHEMA]->(:Schema)-[:HAS_DATASET]->(:Dataset)-[:HAS_COLUMN]->(:Column)
(:Source)-[:HAS_APP]->(:BIApp {tool, app_id, reload_time})-[:HAS_VISUAL]->(:Visual)-[:SHOWS]->(:Metric)
(:BIApp)-[:READS]->(:Dataset)          (:BIApp)-[:WRITES]->(:Dataset {kind:"qvd"})
(:Fact {kind, trust, ...})-[:ABOUT]->(entity)   (:Fact)-[:EVIDENCED_BY]->(:Evidence)
(:Fact)-[:SUPERSEDES]->(:Fact)                   (:Fact)-[:CONFLICTS_WITH]->(:Fact)
Current view: (:Column)-[:DERIVED_FROM]->(:Column)   (:Metric)-[:COMPUTED_FROM]->(:Column)
```

11.5 Knowledge tools used by the starting stage

register_entity, add_context, link_entities, propose_fact, get_entity, get_lineage, get_context_pack, find_join_paths, flag_conflict, record_feedback (review service only). Agents never write raw graph queries; read-only queries are limited to the join planner.

12. Data access: Dremio

Operation (data_port)	Implementation
List catalog, get schema	Arrow Flight SQL metadata calls (ADBC Flight SQL driver)
Get view definition, wiki, tags	Dremio REST catalog API v3
Run bounded pushdown query	Arrow Flight SQL; results as Arrow; row and time limits
Point-in-time query	Iceberg AT TIMESTAMP / AT SNAPSHOT; snapshots via table_snapshot()
Query history (later)	sys.jobs_recent
Files	FileSource over MinIO; DuckDB and Polars for local processing

12.1 Load governance

- Read-only service account.
- Concurrency cap, per-query timeout and row limits (defaults: 4 concurrent queries, 5-minute timeout, to be agreed with the client).
- Every query labelled so the client can see ADCE's load in Dremio's job history.
- Validation batches scheduled off-peak where required.
- No Reflections or other Dremio objects created without client approval.
- Results cached per dataset version; nothing re-queried unless inputs change.

12.2 SQL dialect

All generated SQL targets Dremio SQL and is parsed, validated and generated with sqlglot's Dremio dialect. Other target dialects can be produced later by transpiling.

13. AI model usage

All AI calls go through the **model-gateway** service. Agents never call a model provider directly.

Route	Model	Effort	Used by
Unmapped script statements, T2 expressions	Claude Sonnet 5.5	low–medium	28, 29
T3 expressions, SQL repair, mismatch diagnosis	Claude Opus 5.5	medium–high	25, 26, 29

Route	Model	Effort	Used by
Report export parsing (PDF/vision)	Claude Sonnet 5.5 (Opus for complex layouts)	medium	21
Ambiguous file classification (later)	Claude Haiku 4.5	none	1

13.1 Gateway capabilities

- **Provider:** Claude models through Google Cloud Vertex AI (Gemini reachable through the same adapter interface; other providers for client installs).
- **Structured outputs** for every route, validated against schemas; no free-text parsing.
- **Prompt caching** of stable context (rule tables, schemas, examples).
- **Memoization** keyed by (input hash, model, prompt version): the same inputs regenerate the same outputs, so migration packs are reproducible.
- **Record/replay mode** for tests and CI: no network and no token spend.
- **Budgets** per task and per run, with retry caps.
- **Refusal fallback** through client-side SDK middleware.
- **Tracing** of prompt version, model, tokens, cost and latency to Langfuse.

13.2 Notes for Vertex AI

On Vertex AI, structured outputs, PDF input, prompt caching and token counting are available. The Anthropic Message Batches API, server-side refusal fallbacks and task budgets are not, so bulk work runs as rate-limited concurrent calls, fallbacks run client-side and budgets are enforced by the gateway. Pricing follows Google's rates and is estimated in the first two weeks.

13.3 Privacy policy for this stage

Models may see script text and BI IR, Dremio schemas and view definitions, aggregate differences and report layouts and figures. They never see Dremio rows, QVD rows or sample values. The gateway rejects row-shaped payloads on these routes. Masking with Microsoft Presidio is added when later stages need samples.

14. Human in the loop

Mode	Used for in the starting stage
Ask	Unresolved variables, ambiguous semantics (an agent cannot proceed)
Queue	Source mappings, untranslated statements, conflicts (non-blocking, batch review)
Gate	Approving final SQL; confirming application scope and QVD linkage
Seed	Connection mapping, known variable values
Correct	Reviewer edits a translation; the edit becomes a confirmed fact

Review items are stored as data, so the pipeline never blocks on a person; a decision emits an event that re-runs only the affected steps. The queue is ordered by impact (how many facts depend on an item) times uncertainty. Every decision is stored with the original proposal and reused as an example.

14.1 Workbench screens

Screen	Content
Application overview	Inputs received, statement coverage, validation level per table and figure, open items
Review queue	Ask, Queue and Gate items; batch actions; filters by application and level
Lineage view (from week 6)	Dremio datasets → Qlik tables → QVDs → applications → visuals, each with its validation level

15. Orchestration and events

Events are written to an outbox table in the same transaction as the facts they describe, then published. In the starting stage Postgres LISTEN/NOTIFY wakes consumers; Redis Streams replaces it in later stages. Every handler is idempotent (fact IDs are stable), so retries are safe.

Event	Emitted by	Consumed by
upload.classified[bi_app]	Intake router (1)	BI extractor (27)
bi_inputs.received	API gateway	Orchestrator (re-entry)
dataset.registered / dataset.changed	Scope registrar (5), Schema parser (2)	Resolver, grain check
bi_ir.ready	BI extractor (27)	Script and expression translators (28, 29)
bi_transform.ready / bi_metric_def.ready	Translators	Metric mapper (22), composer (24)
sql.candidates / sql.checked	Synthesizer (24) / Query checker (25)	Checker / Reconciler (26)
fact.proposed / fact.confirmed / fact.rejected	kg-mcp / hitl-service	Orchestrator, affected agents
conflict.flagged	kg-mcp, Join planner (23)	Review queue
pack.published	Publisher (20-lite)	Notifications

16. Contracts

Six versioned contracts are frozen in weeks 1–2 and published as Pydantic models and JSON Schema in a shared core library. Every component and the mock knowledge service are tested against them.

16.1 AgentSpec

```
class AgentSpec(BaseModel):
    name: str; version: str; group: str            # e.g. "bi_extractor", "1.0.0", "reversesql"
    kind: Literal["deterministic", "hybrid", "llm"]
    subscribes: list[str]                           # event types
    allowed_tools: list[str]                        # enforced by kg-mcp and Keycloak scopes
    emits_fact_kinds: list[str]
    hitl_modes: list[Literal["seed", "ask", "gate", "queue", "correct"]]
    model_route: str | None                         # gateway route name
    budget: Budget                                  # tokens, cost, retries per task and per run
```

16.2 BI IR (tool-neutral BI model)

```
class BIApp(BaseModel):
    app_id: str; tool: Literal["qlik_sense", "qlikview", "powerbi", "tableau"]
    name: str; reload_time: datetime | None
    variables: list[BIVariable]          # name, definition, resolved_value, source
    script: BIScript | None              # ordered statements
    model: BIModel                       # tables, fields, association keys, synthetic keys
    master_measures: list[BIMeasure]; master_dimensions: list[BIDimension]
    visuals: list[BIVisual]
    upstream_artifacts: list[ArtifactRef]    # QVDs and other apps this app reads

class BIScriptStatement(BaseModel):
    seq: int; kind: StatementKind        # section 7.2
    target_table: str | None; source: SourceRef | None
    raw_text: str; source_dialect: str | None
    fields: list[BIFieldExpr]; where: ExprAST | None; group_by: list[str] | None
    span: TextSpan                       # file, line, column: used as evidence

class BIMeasure(BaseModel):
    id: str; label: str; description: str | None
    expression_raw: str; expression_ast: ExprAST | None
    construct_tier: Literal["T1", "T2", "T3"]
```

16.3 Ports

```
class DataPort(Protocol):                    # implemented by DremioSource, FileSource, FixtureSource
    def list_catalog(self, scope) -> list[DatasetRef]: ...
    def get_schema(self, dataset) -> Schema: ...
    def get_view_definition(self, dataset) -> str | None: ...
    def query(self, sql, *, at: Snapshot | None, limits: Limits) -> pa.Table: ...
    def query_history(self, since) -> list[JobRecord]: ...          # later stage

class KGPort(Protocol):                      # implemented by the fact-log engine and MockKG
    def register_entity(self, entity) -> EntityRef: ...
    def propose_fact(self, fact, evidence) -> FactRef: ...
    def supersede(self, fact_ref, new_fact) -> FactRef: ...
    def invalidate(self, fact_ref, reason) -> None: ...
    def get_entity(self, ref, *, as_of=None) -> Entity: ...
    def lineage(self, ref, direction, depth) -> Graph: ...
    def context_pack(self, task, token_budget) -> ContextPack: ...
```

16.4 The other contracts

- **Fact payload schemas**, one per fact kind (section 11.3), validated on every write.
- **Event schemas** (section 15).
- **ExprAST**, the shared expression tree (section 8.4).

17. The migration pack

```
<app-name>__<reload_time>__pack-v<N>/
  manifest.json            inputs received, software and model versions, run id
  sql/
    tables/<qlik_table>.sql        Dremio SQL per Qlik table (CTE chain, QVD producers inlined)
    measures/<measure_id>.sql      per measure, where definitions exist
  lineage/
    lineage.json           Dremio datasets -> Qlik tables -> QVDs -> apps -> visuals (OpenLineage-compatible)
    lineage.md             readable summary including QVD chains
  validation/
    report.md, report.xlsx validation level per table and figure, diffs, tolerances, drift reasons
  review/
    untranslated.csv       statements flagged untranslated, with spans
    decisions.csv          reviewer decisions applied
    open-items.md          anything not resolved, e.g. a missing producer application
```

The pack is reproducible: the same inputs and versions regenerate it identically, because AI routes are memoized and every other step is deterministic.

18. Security, privacy and identity

- **Read-only access** to Dremio and Qlik artifacts through dedicated service accounts; ADCE never writes to Dremio and never reloads, saves or changes Qlik applications.
- **No row-level data to AI models** in this stage (section 13.3).
- **Identity:** Keycloak issues user sessions (single sign-on, federating to the client's identity provider later) and a service account per component; each agent's token is limited to its allowed knowledge tools.
- **Roles:** viewer, reviewer, data owner, administrator; only the review service can confirm facts.
- **Isolation:** each client is a separate tenant, enforced inside the knowledge store; queries without a tenant binding are refused.
- **Secrets** from the environment's secret store (Google Secret Manager in our environments).
- **Audit:** the fact log is append-only and records every proposal, decision, prompt version and model version; any past state can be reconstructed.
- **Fail closed:** missing configuration, missing authentication or a missing tenant binding stops the service rather than degrading silently.

19. Environments and deployment

Environment	Runs on	Data	AI models
Local	Docker Compose with profiles: core (Postgres, MinIO, kg-mcp, model-gateway, API), agents, ui, auth (Keycloak), obs (Langfuse), dremio-local (Dremio OSS + sample data), bi-fixtures (sample Qlik inputs). Windows, macOS and Linux.	Local Dremio with a sample dump; fixtures	Development Vertex project, or record/replay
Staging (from week 9)	Google Cloud: GKE, Cloud SQL for Postgres, Cloud Storage, Secret Manager, Neo4j self-hosted	Non-production Dremio, read-only	Staging Vertex project
Production / client	The same Helm chart: hosted by us in a dedicated project per client, or installed in the client's environment	Client Dremio, read-only	Client's approved endpoint

19.1 Built to ship from day 1

- Every external dependency behind an interface selected by configuration.
- One configuration schema, validated at startup.
- Forward-only automatic migrations (Alembic for Postgres, versioned Cypher for Neo4j), tested on the upgrade path.
- Container images built once and promoted unchanged from local to staging to production.
- Signed images with a software bill of materials; vulnerability scanning in CI.
- A support bundle with diagnostics and redacted configuration, never data.
- Air-gap capable: no runtime downloads and no outbound calls except the configured AI endpoint.

19.2 CI/CD

Every change runs lint, type checks, unit tests, contract tests against the mock knowledge service, and the gold-set evaluation when prompts, models or translation rules change. Promotion to staging and production is gated by the gold-set results, contract tests and security scans.

20. Observability and evaluation

- **Langfuse** (self-hosted) traces every AI call with prompt version, model, tokens, cost and latency, and stores evaluation datasets.
- **OpenTelemetry** traces across services; metrics and dashboards for queue depth, Dremio query load, pipeline durations and failures.
- **Diagnostics** show what is queued, what failed, and whether it will retry or fail again.
- **Running-version fingerprint** in health checks and every pipeline result.

20.1 The gold set

Assembled in weeks 1–2 and grown throughout: 2–3 Qlik applications with scripts, reload logs, QVD files and exported reports for at least 10 visuals spanning T1–T3, with hand-verified SQL for the T1–T2 tables and visuals and evidence captured at a fixed reload time. Every change to prompts, models or rules is evaluated against it, and results are reported as ranges over repeated runs, never single runs.

21. Technology stack

Layer	Choice	Notes
Languages	Python 3.12 (services and agents); TypeScript (workbench)	uv, ruff, pyright, pytest; pnpm, ESLint, Playwright
Workbench	Next.js with React Flow	Three screens in this stage
API and identity	FastAPI; Keycloak	OIDC/SAML single sign-on; service accounts
Orchestration	LangGraph 1.x with the Postgres checkpointer	Interrupts only for Ask
Knowledge tools	FastMCP 3.x (kg-mcp)	Typed tools, OAuth, OpenTelemetry
Knowledge store	Postgres 16 (fact log, outbox, review items) with pgvector; Neo4j 5.26 LTS projection with APOC	Built on our in-house knowledge-graph engine
AI models	Claude Opus 5.5, Sonnet 5.5, Haiku 4.5 on Google Vertex AI (anthropic[vertex]); Gemini via google-genai	Through model-gateway only
Data source	Dremio via the ADBC Flight SQL driver and the REST catalog API v3	Read-only; Iceberg time travel
Local compute	DuckDB 1.5.x, Polars, Apache Arrow	Files and small extracts
SQL	sqlglot (Dremio dialect)	Parsing, transpiling, validation, rule checks
Qlik parsing	Lark grammar; lxml for QlikView -prj XML; reload-log parser	Grown from real scripts
QVD files	qvdrs (Rust, Python bindings); PyQvd as fallback	Header lineage and data
Report exports	openpyxl (Excel); Claude PDF/vision input (PDF)	Figures only
Object storage	MinIO locally; Google Cloud Storage via the S3-compatible API	Inputs, BI IR, packs
Events	Postgres outbox with LISTEN/NOTIFY; Redis Streams later	Idempotent handlers
Observability	Langfuse v3, OpenTelemetry, Grafana/Prometheus	
Privacy (later)	Microsoft Presidio	When samples are needed

Layer	Choice	Notes
Exports	Jinja templates, openpyxl, OpenLineage JSON	Migration pack
Deployment	Docker Compose (local); Kubernetes with Helm on GKE (staging, production)	Same images everywhere
Migrations and release	Alembic, versioned Cypher migrations, semver, signed images, SBOM	
CI	GitHub Actions	Lint, types, tests, contract tests, gold-set evaluation

22. Repository layout

```
agentic-data-context-engine/
  docs/                       architecture, decisions, proposals, open questions
  packages/
    adce-core/                AgentSpec, BaseAgent, fact and event schemas, clients
      bi_ir/                  BI IR models and ExprAST
      kg_port/                knowledge port + MockKG
      data_port/              DremioSource, FileSource, FixtureSource
    kg-engine/                knowledge core (fact log, projection, isolation, search, resolution)
    qlik/                     statement splitter, Lark grammar, BI IR builder, reload-log parser, QVD reader
  services/
    kg-mcp/                   knowledge tools, policy, validator, gate, context packs
    model-gateway/            routes, budgets, memoization, replay, privacy, tracing
    hitl-service/             ask, queue, gate, decisions, examples
    orchestrator/             LangGraph supervisor, qlik_pipeline subgraph, event consumers
    api-gateway/              uploads, exports, auth
    agents-ingestion/         intake_router, schema_parser, entity_linker, scope_registrar
    agents-bi-adapters/       bi_extractor, bi_script_translator, bi_expression_translator
    agents-reversesql/        report_parser, metric_mapper, join_planner, sql_synthesizer,
                              query_checker, reconciler
    agents-profiling/         key_discoverer (grain, lite)
    agents-definition/        lineage_inferrer (lite), dictionary_publisher (migration pack)
    agents-comparison/        equivalence_detector (lite)
  apps/workbench/             Next.js workbench
  prompts/                    versioned prompts per agent
  eval/gold/bi/               apps/<app>/{scripts,logs,qvd,exports}, expected/<table>.sql, expected/figures.csv
  deploy/                     compose profiles, helm chart, config schema, keycloak realm
  migrations/                 alembic, cypher, upgrade-path tests
```

23. Delivery plan

Phase	Weeks	Work packages	Done when
Foundations	1–2	Repository, CI and configuration schema; local environment with Postgres, MinIO, Keycloak, Langfuse and Dremio OSS with a sample dump; six contracts frozen; fact log, outbox and knowledge port; model gateway on Vertex with replay and the privacy check; Qlik statement splitter and grammar on every sample application; QVD header reader; reload-log parser; connection mapping drafted; gold set v0	Sample applications parse with measured coverage; one simple Qlik table translated and executing in local Dremio (V2)
Tables	3–5	Complete extraction for script inputs; script translation for T1 then T2; source resolution including QVD chains; grain checks; Query-checker grain and fan-out rules; reconciliation V2–V4 with caching and time travel; review queue and Ask in the workbench; all results in the fact log	At least 90% of statements translated; at least 80% of gold tables at V3 or higher; every unresolved item reaches review

Phase	Weeks	Work packages	Done when
Figures and lineage	6–8	Expression translation T1–T2; export parsing; composition per visual; set-analysis, TOTAL and Aggr rules; V5 reconciliation; diagnoser; graph projection and lineage view	At least three real report figures at V5; lineage across QVD chains browsable
Harden and package	9–12	More applications; T3 attempts; migration pack export; equivalence checks where Qlik reads copies; Dremio load tuning; staging on Google Cloud; engine adapter if server access exists	Targets in section 24 met; a migration pack accepted by a report owner; the same build running in staging

23.1 Weeks 1–2 in detail

Work package	Week 1	Week 2
Qlik parsing	Statement splitter on all sample scripts; statement-kind classifier; first grammar for LOAD, SQL SELECT, STORE, JOIN, CONCATENATE; QVD header reader	Grammar for MAPPING LOAD, ApplyMap, RESIDENT, preceding load; reload-log parser; coverage report per application; one T1 table translated
SQL correctness	ExprAST and BI IR contracts; Dremio-dialect tests in sqlglot; checker skeleton	Reconciler V2/V3 skeleton; first V2 run in local Dremio
Dremio access	data_port interface; DremioSource on local Dremio OSS	Catalog crawl and view definitions as facts; connection mapping drafted
Grain	Pushdown uniqueness query library	Grain facts for datasets in the sample scripts
Lineage and pack	Lineage fact shapes from transforms, QVD headers and view definitions	Migration-pack layout and exporter skeleton
Knowledge core	Repository, CI, configuration schema, Compose profiles	Fact log, outbox, knowledge port, MockKG
Gateway and review	Gateway on Vertex with structured outputs, memoization, replay and the privacy check	Review service tables and API; qlik_pipeline skeleton
Workbench	Scaffold, sign-in, API gateway skeleton	Application overview screen
Gold set	Collect inputs for 2–3 applications	Hand-verify expected results for T1 tables; evaluation harness reporting levels

24. Success measures

Measure	Target by week 12
Script statements translated without being flagged	At least 90% per application
Gold tables at V3 or higher	At least 80%
Gold tables at V4, where QVD files exist	At least 70%
Report figures at V5 (T1–T2), where exports exist	At least 70%
Results shown with their validation level and tolerance	100%
Unresolved variables or sources silently assumed	None
Migration packs regenerated identically from the same inputs	100%
Cost per application translated	Measured and reported

25. Risks and mitigations

Risk	Mitigation
Unusual Qlik syntax	Grammar grown from real scripts; fallback to flagged statements plus AI; coverage tracked per application
Only load scripts provided	Table-level SQL and lineage still delivered; figure-level work starts when project folders or app files arrive
Data changed since the last reload	Time travel where available; stated tolerances otherwise
Qlik Cloud reload logs are summaries	Client-managed script logs where available; QVD headers and data as fallback
Qlik reads copies or sources outside Dremio	Reviewed connection mapping; equivalence checks; reproduce what Qlik reads and flag the golden source
Missing producer applications in QVD chains	Not resolved, with the missing application named
Silent double counting	Grain-aware translation, Query-checker rule, multi-slice reconciliation
Load on the client's Dremio	Bounded, cached, labelled, rate-limited and off-peak queries
AI output varies between runs	Structured outputs, pinned prompt and model versions, memoization, regression evaluation
Lean components cause rework	Final interfaces from day 1; the fact log is the target system of record
Cost overruns	Per-task and per-run budgets, caching, routing by difficulty

26. Open questions

Question	Why it matters
Which Qlik inputs are provided (scripts, project folders, app files, reload logs)?	Decides whether figure-level reconstruction is possible from the start
Will QVD files be provided?	Unlocks V3/V4 and cross-application lineage
Are reload logs detailed (client-managed) or summaries (Qlik Cloud)?	Row-count validation source
Which Qlik data connections exist and what do they point to?	Source resolution
Is the shaping SQL in Dremio views, inside the scripts, or both? Do scripts read anything outside Dremio?	Where lineage comes from
Are the relevant Dremio tables Iceberg?	Exact point-in-time comparisons
Dremio edition, version and permissions; load limits	Connector design and governance
Google Cloud project, region and model quota	Model access
Exported reports or reference figures for 10 or more visuals	Figure-level validation
Report and data owners available for review	Confirmations and approvals

27. What follows the starting stage

From week 6 the wider platform builds on the same foundations: profiling of Dremio datasets in place, sensitive-data tagging, key and join discovery, detection of overlapping and duplicate datasets with recommended golden records, the data dictionary, reconstruction of reports for which no BI logic exists, and live Qlik engine validation once server access is granted. At a later stage, everything learned is used

to propose a designed data model (a star schema, a normalized business model or a semantic layer of curated Dremio views), proven by reproducing existing reports. The overall proposal describes the complete product and roadmap.

28. References

[1] Nagarajan, A. and Altman, T. DBAutoDoc. arXiv:2603.23050, 2026.

[2] Tang, Z. et al. LLM-FK. Findings of ACL 2026. arXiv:2603.07278.

[3] Tran, Q. T., Chan, C.-Y. and Parthasarathy, S. Query Reverse Engineering. The VLDB Journal, 2013.

[4] Qlik community documentation: QVD XML header (CreatorDoc, NoOfRecords, LineageInfo).

[5] Qlik developer changelog: reload log format changes (2026).

[6] PyQvd and qvdrs projects (QVD reading in Python and Rust).

[7] Dremio documentation: Arrow Flight SQL, REST catalog API v3, system tables, Iceberg time travel (AT SNAPSHOT, AT TIMESTAMP).

[8] sqlglot documentation: dialects, including Dremio.

[9] LangGraph documentation: interrupts and the Postgres checkpointer.

[10] FastMCP 3.x documentation.

[11] Anthropic documentation: Claude on Google Cloud Vertex AI, structured outputs, prompt caching.

[12] Langfuse self-hosting documentation.